Week 4: OLS Demand Model

```
import numpy as np
import pandas as pd
pd.options.mode.chained_assignment = None
import pickle

import os
from importlib import reload

import matplotlib.pyplot as plt
import matplotlib
# import seaborn as sns
%matplotlib inline
plt.style.use('ggplot')
from matplotlib.ticker import MultipleLocator
```

load competition data and show some stats

```
df_comp_details = pd.read_csv('duopoly_competition_details.csv')
df_comp_details.fillna(0, inplace=True)

df_comp_details['revenue'] = df_comp_details['price'] *
df_comp_details['demand']

df_comp_details.head(3)
```

```
  competition_id  selling_season  selling_period   competitor_id  \
0         3C4dDe               1               1  LoutishMacaque
1         3C4dDe               1               2  LoutishMacaque
2         3C4dDe               1               3  LoutishMacaque

   price_competitor  price  demand  competitor_has_capacity  \
0              56.1   56.3       0                     True
1               2.0   56.3       0                     True
2               3.0   56.3       0                     True

   calculation_duration  errors  revenue
0                 0.004     0.0      0.0
1                 0.004     0.0      0.0
2                 0.003     0.0      0.0
```

```
df_comp_details['unique_selling_season_key'] =
df_comp_details.apply(lambda r:
                                "%s_%s" %
(r.competition_id,r.selling_season), axis=1)

dfx_rev = df_comp_details.groupby('unique_selling_season_key').agg({
    'revenue' : 'sum'
```

```
}).reset_index()
dfx_rev.head(3)
```

```
  unique_selling_season_key  revenue
0                   3C4dDe_1    115.9
1                  3C4dDe_10    300.8
2                 3C4dDe_100    337.3
```

Let us build our own demand model based on the competition data.

=> f(price, competitor_price) = demand

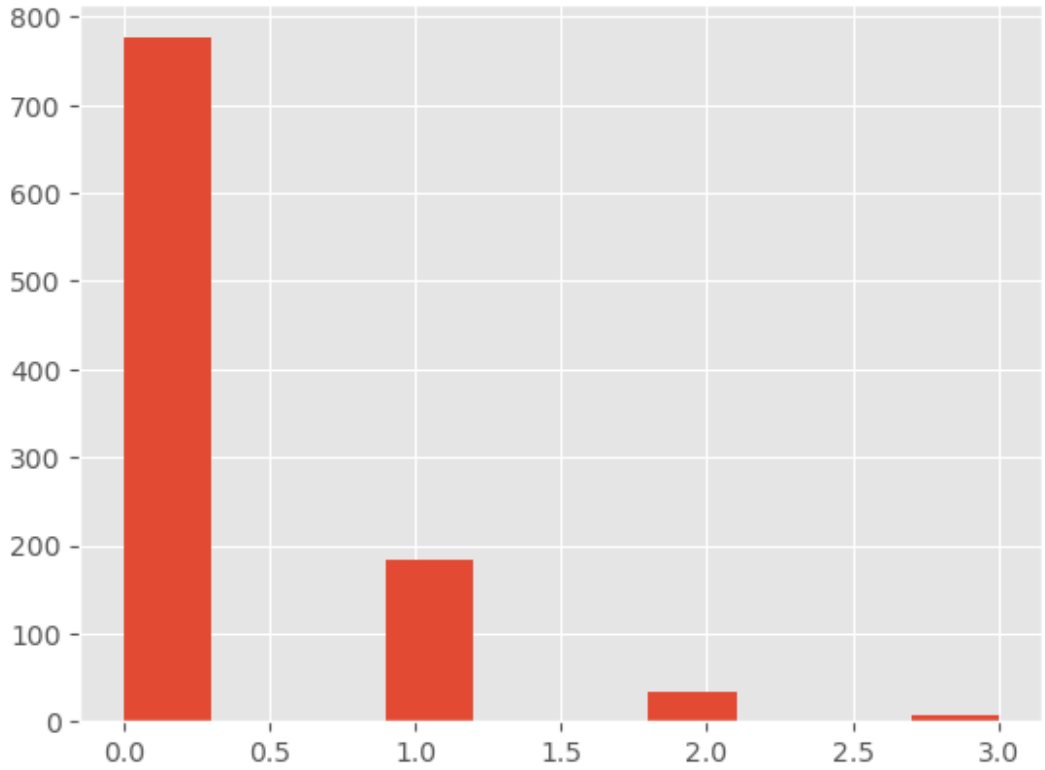
idea:

- split selling_season into sub-intervals where we assume a constant demand model
  - e.g. selling period [1,20] assumed to have almost constant demand
- estimate the demand model per competition
- train on first 50 selling_seasons and evaluate on the last 50

```
comp = df_comp_details.competition_id.unique()[1]
print(comp)

# filter on first competion_id
df = df_comp_details[df_comp_details.competition_id==comp ]

# concentrate on selling period in [1,20]
df = df[df.selling_period <= 20 ]

df_train = df[df.selling_season<=50]
df_test = df[df.selling_season>50]

df_train.demand.hist()
```

```
3yUFj3
```

```
<Axes: >
```



(a) fit OLS model

```
import statsmodels.api as sm
#
https://www.statsmodels.org/stable/generated/statsmodels.regression.linear_model.OLS.html

y = df_train.demand
X = df_train[['price', 'price_competitor']] #, 'selling_period']]
X['intercept'] = 1

mod = sm.OLS(y, X)
# mod = sm.regression.quantile_regression.QuantReg(y,X)

res = mod.fit()

print(res.summary())
```

```
                            OLS Regression Results

==============================================================================
Dep. Variable:                 demand   R-squared:
0.124
```

```
Model:                            OLS   Adj. R-squared:
0.122
Method:                 Least Squares   F-statistic:
70.62
Date:                Tue, 28 Oct 2025   Prob (F-statistic):
2.07e-29
Time:                        00:24:22   Log-Likelihood:
-759.89
No. Observations:                1000   AIC:
1526.
Df Residuals:                     997   BIC:
1540.
Df Model:                           2

Covariance Type:            nonrobust

==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
price         -0.0100      0.001    -10.234      0.000      -0.012      -0.008
price_competitor     0.0074      0.001      8.274      0.000       0.006       0.009
intercept            0.3139      0.040      7.908      0.000       0.236       0.392
==============================================================================
Omnibus:                      396.184   Durbin-Watson:
2.000
Prob(Omnibus):                  0.000   Jarque-Bera (JB):
1538.252
Skew:                           1.893   Prob(JB):
0.00
Kurtosis:                       7.752   Cond. No.
118.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is
correctly specified.
```

(b) predict demand based on function for

- own price = 30

- competitor price = 50

```
mod.predict(res.params, [30, 50, 1])

np.float64(0.38279312992154885)
```

(c) simulate demand, our prices in [1,100] and competitor_price = 50

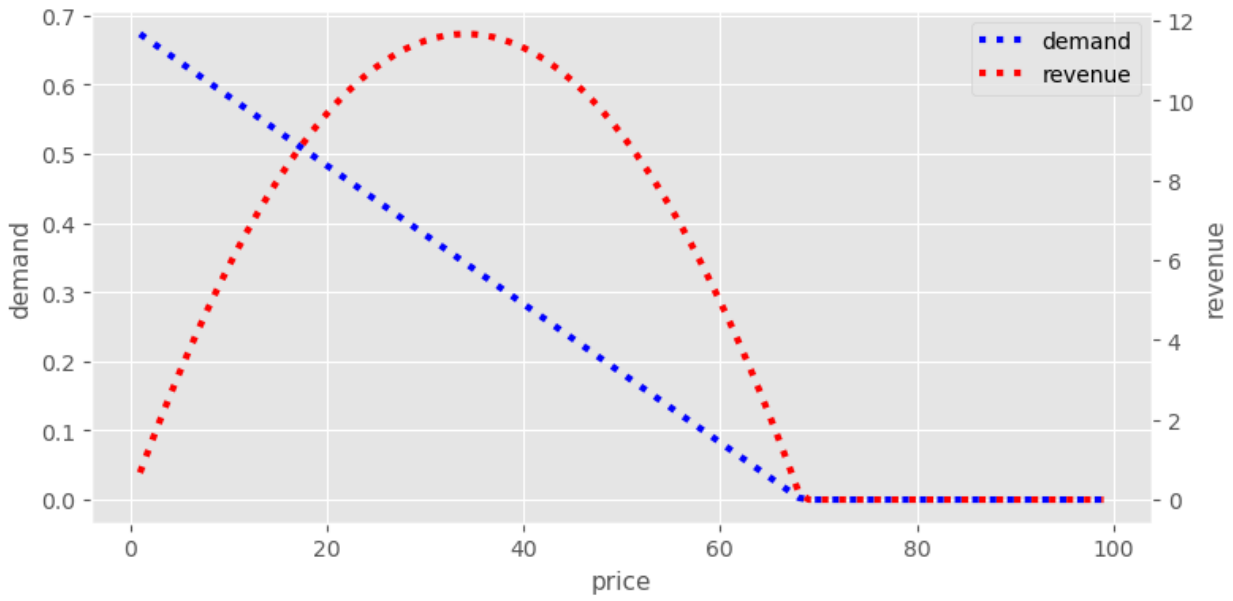
```
def est_demand(price, comp_price, para = res.params):
    demand = mod.predict(para, [price, comp_price, 1])
    demand = max(0,demand)
    return demand


df_sim = pd.DataFrame(columns = ['price','demand','rev'])
for p in np.arange(1,100,1):
    demand = est_demand(price=p, comp_price=50)
    rev = p * demand
    df_sim.loc[len(df_sim)] = [p,demand,rev]

df_sim.head()
```

	price	demand	rev
0	1.0	0.672887	0.672887
1	2.0	0.662883	1.325767
2	3.0	0.652880	1.958640
3	4.0	0.642877	2.571508
4	5.0	0.632874	3.164368

plot demand and revenue curve

```
fig, ax = plt.subplots(figsize=(8,4))

ax.plot(df_sim['price'], df_sim['demand'], color='b',
        linestyle='dotted', linewidth=3, label='demand')

ax_2 = ax.twinx()
ax_2.grid(False)
ax_2.plot(df_sim['price'], df_sim['rev'], color='r',
        linestyle='dotted', linewidth=3, label='revenue')

ax_2.tick_params(axis="both", labelsize=10)
ax.tick_params(axis="both", labelsize=10)

# legend
lines, labels = ax.get_legend_handles_labels()
```

```
lines2, labels2 = ax_2.get_legend_handles_labels()
ax_2.legend(lines + lines2, labels + labels2, loc=0)

ax.set_xlabel('price', size=11)
ax.set_ylabel("demand" , size=11)
ax_2.set_ylabel("revenue" , size=11)

ax.grid(True)
fig.tight_layout()

plt.show()
```



(d) compute estimated demand in computation details

```
df_eval =
df_test[['unique_selling_season_key','selling_season','selling_period'
,'price_competitor', 'price', 'demand']]

df_eval.head(5)
```

	unique_selling_season_key	selling_season	selling_period	\
15000	3yUFj3_51	51	1	
15001	3yUFj3_51	51	2	
15002	3yUFj3_51	51	3	
15003	3yUFj3_51	51	4	
15004	3yUFj3_51	51	5	

```
       price_competitor  price  demand
15000              57.5   56.4       0
15001              42.0   56.4       1
15002              42.0   56.4       0
15003              42.0   56.4       0
15004              42.0   56.4       0
```

```
# estimate demand
df_eval['est_demand'] = df_eval.apply(lambda r:
            est_demand(price=r.price, comp_price=r.price_competitor),
            axis=1)

# compute error metrics
df_eval['est_error'] = df_eval['est_demand'] - df_eval['demand']
df_eval['abs_est_error'] = df_eval['est_error'].abs()

df_eval.head(5)
```

```
      unique_selling_season_key  selling_season  selling_period  \
15000                 3yUFj3_51              51               1
15001                 3yUFj3_51              51               2
15002                 3yUFj3_51              51               3
15003                 3yUFj3_51              51               4
15004                 3yUFj3_51              51               5

       price_competitor  price  demand  est_demand  est_error
abs_est_error
15000              57.5   56.4       0    0.174058   0.174058
0.174058
15001              42.0   56.4       1    0.059668  -0.940332
0.940332
15002              42.0   56.4       0    0.059668   0.059668
0.059668
15003              42.0   56.4       0    0.059668   0.059668
0.059668
15004              42.0   56.4       0    0.059668   0.059668
0.059668
```

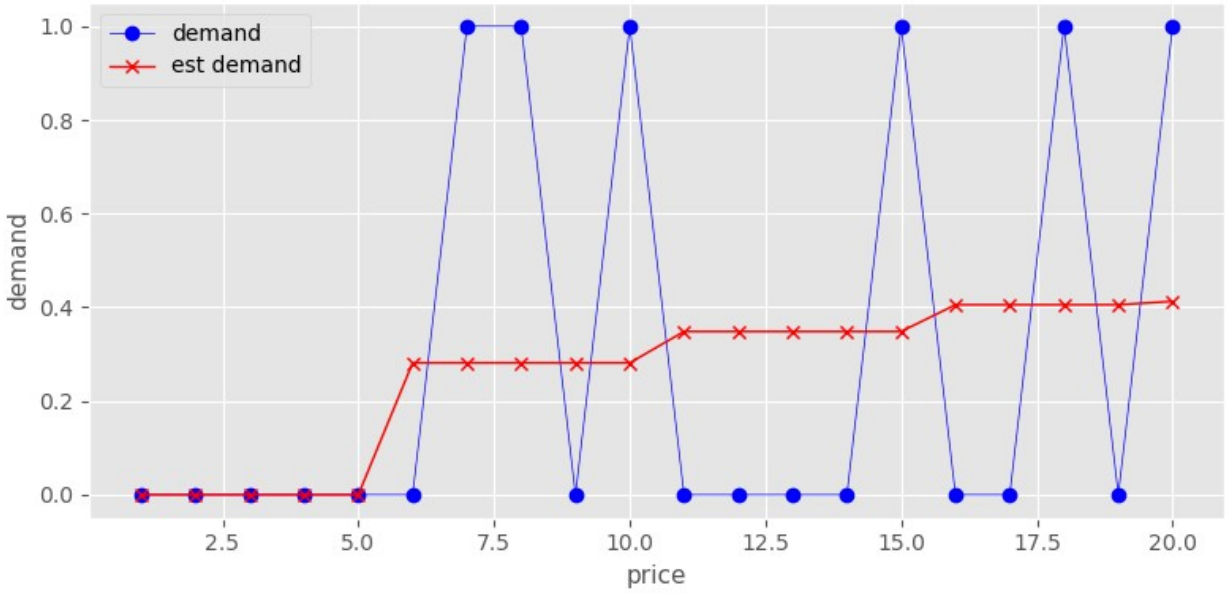
```
df_eval[['demand', 'est_demand', 'est_error',
'abs_est_error']].describe()
```

```
            demand   est_demand    est_error  abs_est_error
count  1000.000000  1000.000000  1000.000000    1000.000000
mean      0.354000     0.341766    -0.012234       0.462066
std       0.636466     0.162889     0.613676       0.403769
min       0.000000     0.000000    -3.537160       0.000000
25%       0.000000     0.255108    -0.491438       0.261940
```

```
50%       0.000000     0.392818     0.220608       0.423695
75%       1.000000     0.462840     0.409914       0.540268
max       4.000000     0.643521     0.643521       3.537160
```

plot the actual demand vs. estimated demand for one test selling season

```
dfe = df_eval[df_eval.selling_season==60]

fig, ax = plt.subplots(figsize=(8,4))

ax.plot(dfe['selling_period'], dfe['demand'], color='b',
        linestyle='-', marker='o', linewidth=0.5, markersize=6,
label='demand')

ax.plot(dfe['selling_period'], dfe['est_demand'], color='r',
        linestyle='-', marker='x', linewidth=1, markersize=6,
label='est demand')

ax.legend( loc=0)

ax.set_xlabel('price', size=11)
ax.set_ylabel("demand" , size=11)

ax.grid(True)
fig.tight_layout()

plt.show()
```



followup tasks:

(1) estimate five demand models covering the 100 period selling_season

(2) estimate demand models for every simulation_id

(3) implement the demand model in your own simulation (week 2)

(4) implement the demand model estimation & next price optimization in your pricing algorithm